1. Deep Q-Network:

The Deep Q-Network (DQN) is a groundbreaking approach in reinforcement learning that combines Q-learning, a form of model-free reinforcement learning, with deep neural networks. DQN was able to achieve human-level performance on many Atari 2600 games using raw pixels as input, marking a significant advancement in the field of artificial intelligence.

Core Components of DQN:

Q-Learning: At the heart of DQN is Q-learning, a value-based reinforcement learning algorithm. Q-learning aims to learn the value of taking a given action in a given state, quantified as Q-values. These values are used to make decisions by selecting the action with the highest Q-value in each state.

Deep Neural Networks: DQN uses a deep neural network to approximate the Q-value function. The network takes the state (e.g., game screen pixels) as input and outputs Q-values for all possible actions. This approach allows DQN to handle high-dimensional state spaces.

Experience Replay: To break the correlation between consecutive samples and to reuse past experiences, DQN stores the agent's experiences at each time step in a replay buffer. Random mini-batches from this buffer are used to train the network, improving sample efficiency and stability.

Fixed Q-Targets: DQN uses a separate target network to generate the Q-value targets for training the main network. The weights of the target network are updated less frequently (e.g., every few thousand steps) to stabilize training. This helps to mitigate the moving target problem, where updates to the Q-values change the targets for future updates.

2. Describe the architecture of your PacmanActionCNN

In def __init__(self, state_dim, action_dim):

self.conv1: The first convolutional layer that takes the input state. It uses 32 filters with a kernel size of 8 and a stride of 4. This layer is responsible for extracting low-level features from the input state.

self.conv2: The second convolutional layer that takes the output from conv1. It uses 64 filters with a kernel size of 4 and a stride of 2, further processing the features extracted by conv1.

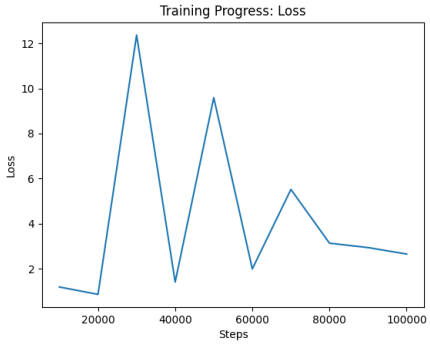
self.fc1: A fully connected (linear) layer that takes the flattened output from conv2 and transforms it into a 512-dimensional vector. This layer serves to interpret the features extracted by the convolutional layers in a high-dimensional space.

self.fc2: The final fully connected layer that maps the 512-dimensional vector from fc1 to the action space dimensionality (action_dim). This layer outputs the Q-values for each possible action.

In def forward(self, x):

The input x is passed through the first convolutional layer (conv1) and a ReLU activation function is applied to introduce non-linearity. The output of conv1 is then passed through the second convolutional layer (conv2), followed by another ReLU activation. The output of conv2 is flattened (i.e., reshaped into a 1D tensor) to match the input requirements of the fully connected layers. The flattened tensor is passed through the first fully connected layer (fc1) and a ReLU activation function. Finally, the output of fc1 is passed through the second fully connected layer (fc2) to produce the final output, which represents the Q-values for each action.

3. Plot your training curve, including both loss and rewards



4. Show screenshots from your evaluation video